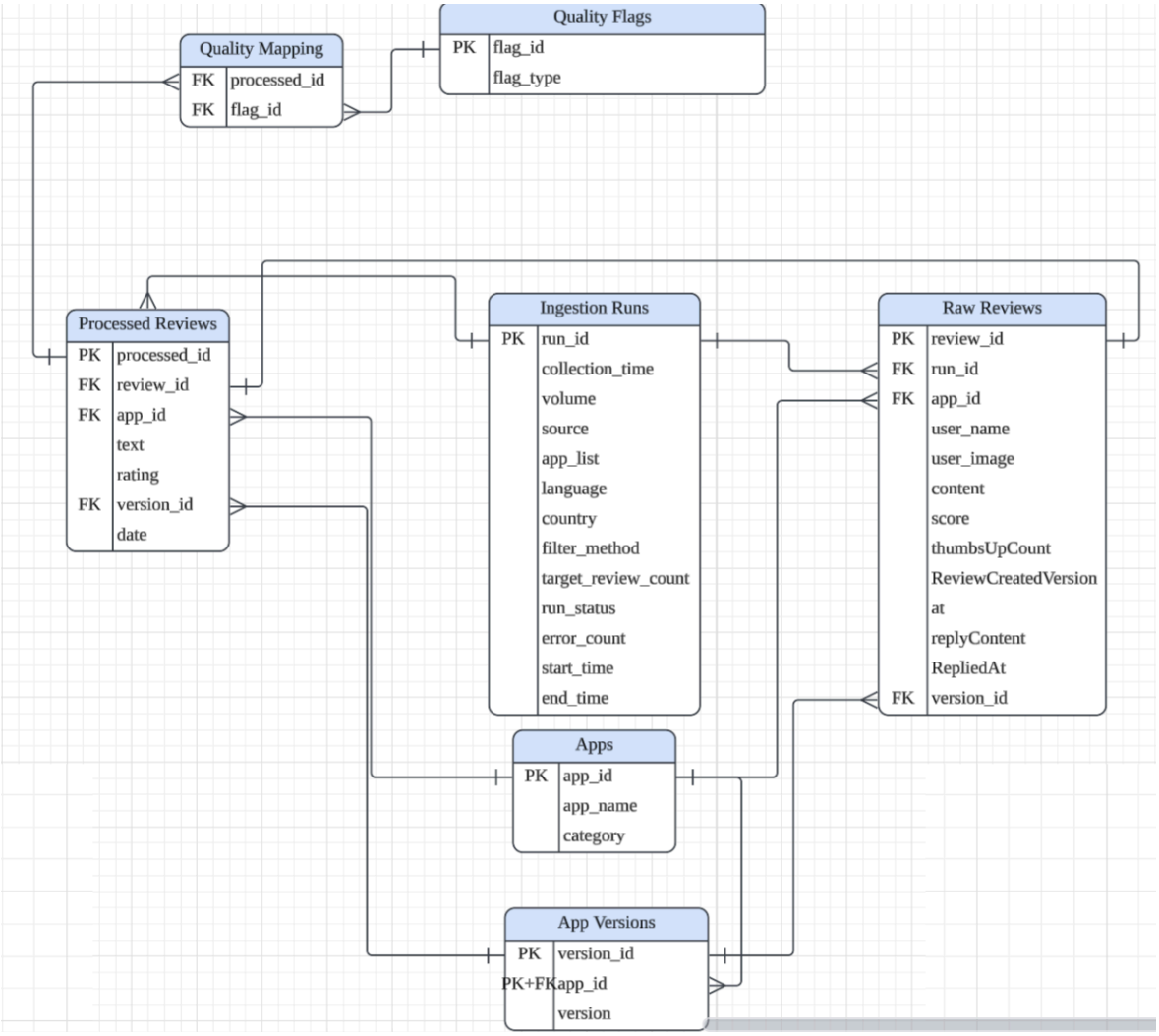

I got 7 tables shown above.

Design Logic:

Apps:

- Stores static metadata of target apps.
- The primary key is app_id, which is also the unique identifier for each app. There is no foreign key in this table.

App Version:

- It shows the app_version for each parent app
- App version should not be treated as globally unique by itself, so we set version_id and app_id together as the composite primary key to uniquely identify versions for each app. The same version can exist across different apps, so we set a composite key to prevent duplication.
- App_id is also the foreign key of this table to connect the apps table.

- The relationship between app and app_version tables is one-to-many relationship because one app can have multiple versions.

Ingestion Runs:

- Tracks metadata of every recurring Google Play data collection job and captures full operational context.
- For each run, we have run_id as the primary key to uniquely identify each run. There is no foreign key inside the table.
- The table covers many operational context for ingestion: collection time, volume, source/platform, app list, language, country, target review count, run status, error count…
- Every record inside the Raw Reviews table links back to its originating run_id, enabling full traceability.

Raw Reviews:

- Preserves unmodified raw review data from Google Play without cleaning or transformation
- The primary key review_id is a unique identifier for each review. And we have 3 foreign keys that connect to tables: AppVersion, App and IngestionRun. All these relationships are one-to-many relationships because one app have many raw reviews, one app version can have many raw reviews, and each run has multiple reviews.
- I separated raw reviews from processed reviews in order to guarantee we retain the original version if cleaning logic needs to be revised in the future.

Processed Reviews:

- Cleaned review dataset optimized for downstream analysis.
- The primary key is processed_id, and the foreign key is review_id that has one-to-one relationship with raw reviews table since each raw review has only cleaned and standardized processed review.

Quality Flags:

- Stores all supported quality defect types, such as duplication, low-signal text, missing fields, non-English content, emoji-only content.
- Primary key flag_id is used to avoid duplicated quality issues. No foreign key in this table
- Instead of adding fixed columns is_duplicated, low_signal inside Processed Reviews table, we use a junction table. New quality issues can be added by inserting new rows here, making it more flexible. If we add it to fixed column, it might be messy and troublesome for great expansion in quality issues.

Quality Mapping:

- Many-to-many junction table connecting cleaned reviews to quality flags. The relationship between the two sides are one-to-many.
- The columns are composed with foreign keys from two tables, forming a composite key for a unique identifier.

Deduplication Logic

1. In order to avoid repeated duplication of versions, we use a composite key (app_id + version_id) to uniquely identify each version for an app
2. Raw data deduplication design: Composite unique key (run_id, app_id, review_id) to eliminate duplicate scraped reviews from the same ingestion run and app.
3. Quality flags: Remove boolean columns (is_duplicated, low_signal, missing text) from Processed Reviews table. All quality issues are tracked through the Quality Flags table and QualityMapping one-to-many table for scalable tagging and detailed issue notes.

Tradeoffs

1. I separate the raw reviews and processed reviews into two tables so we can always access the raw dataset if we encounter any problems with the processed dataset. Even though it takes more space to store and requires JOIN with Raw Reviews table if original unmodified text is needed, I think reproducibility or traceability is more important than storage.
2. I include quality issues in a separate table to make it flexible for tracking and later expansion. Even though this new table adds more complexity for joining tables, flexibility is more important for traceability.
3. Even though it might increase query complexity to JOIN two reviews tables many times with the junction table in the middle, this provides a more clear and maintainable database structure (less repeated rows and keeps the schema lightweight and extensible with a great number of categories)